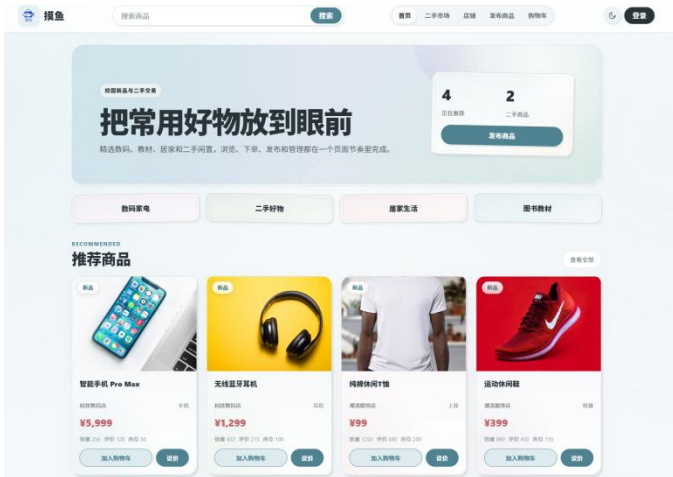
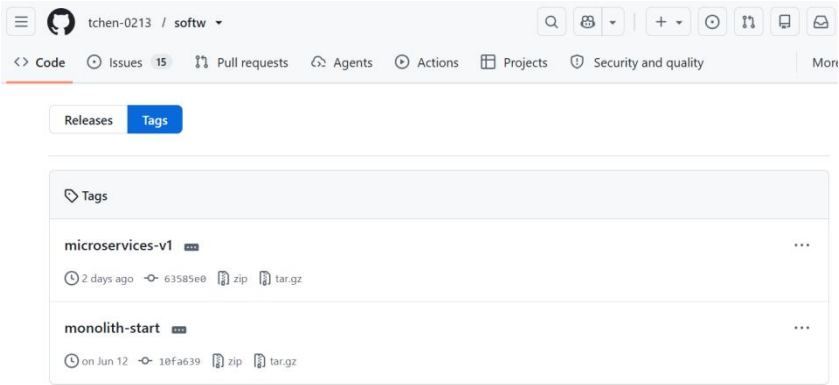
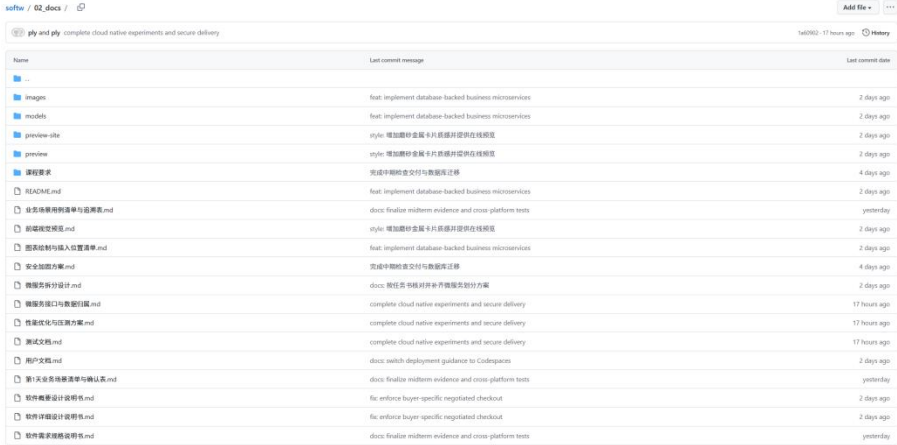
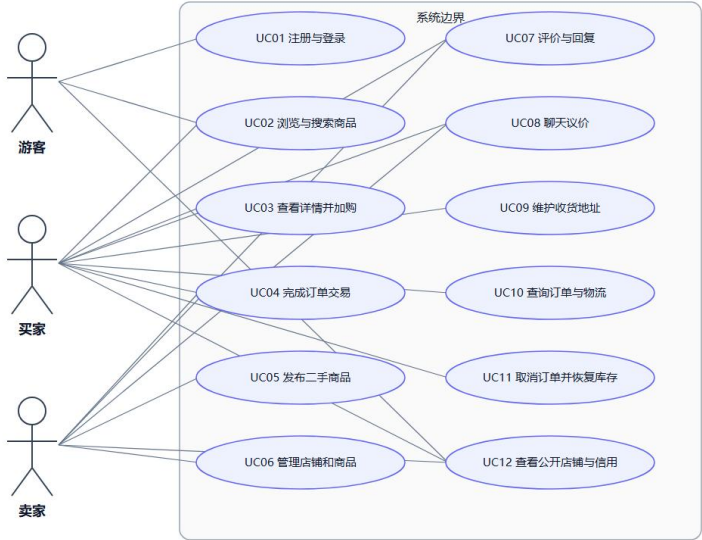


softw 中期审查核对清单

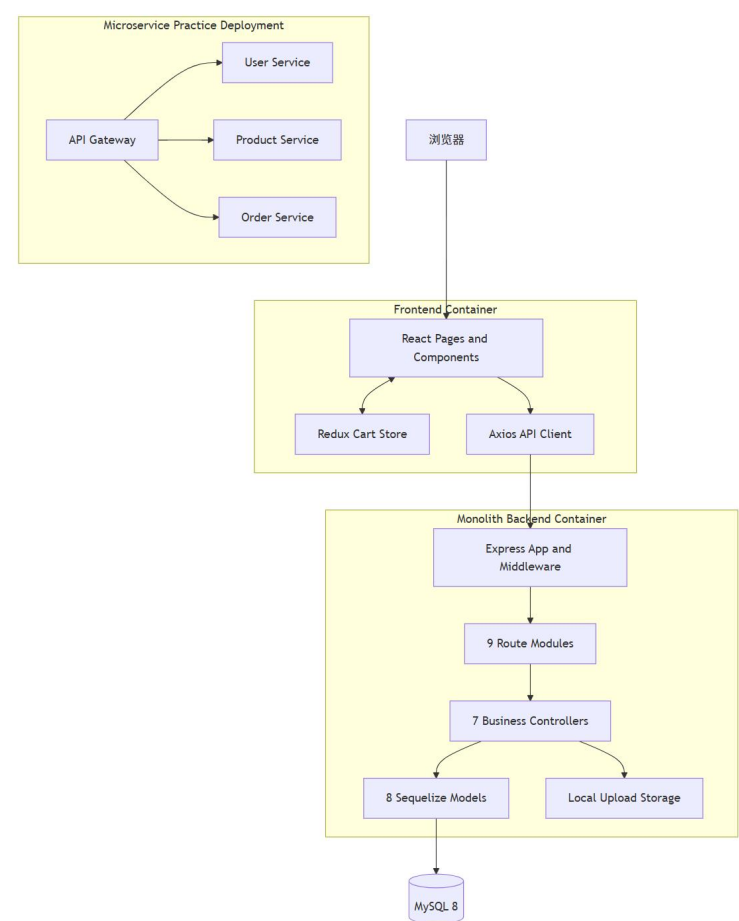
仓库	https://github.com/tchen-0213/softw
生成日期	2026-09-02
依据	GitHub main 分支；代码验收基线 c1b73ec。依据最新测试报告、微服务方案、42 张图清单及 softw-ci-cd #77 成功记录。

一、中期审查总项

审查项	出示或执行的证据	状态												
原系统可启动 证明单体系统仍可复现。	README.md 、 03_devops/docker-compose.yml 、 03_devops/scripts/verify-local.sh ; 可执行 npm run verify 、 npm run compose:up,访问前端和/api/health 。	已对项目原始版本进行启动验证，确认系统能够正常运行。启动后前端页面可正常访问，系统核心运行环境与基本功能保持可用。 已在 GitHub Project [D1-01] 启动原系统并核对运行环境 中完成现场验证并留存截图。截图包含原系统启动、运行环境及访问结果。 单体部署实测记录 (2026-09-02) <table><tr><th>项目</th><th>结果</th></tr><tr><td>Compose</td><td>frontend、backend、mysql 均 Healthy</td></tr><tr><td>前端与健康接口</td><td>首页 HTTP 200; /api/health 返回 ok</td></tr><tr><td>数据库</td><td>真实 MySQL; 3 个版本迁移已应用</td></tr><tr><td>Kind</td><td>3 个工作负载均 1/1; 单体入口 :30080</td></tr><tr><td>回归</td><td>API 32/32; 浏览器 E2E 6/6</td></tr></table> 完整部署与测试证据 首页截图： 	项目	结果	Compose	frontend、backend、mysql 均 Healthy	前端与健康接口	首页 HTTP 200; /api/health 返回 ok	数据库	真实 MySQL; 3 个版本迁移已应用	Kind	3 个工作负载均 1/1; 单体入口 :30080	回归	API 32/32; 浏览器 E2E 6/6
项目	结果													
Compose	frontend、backend、mysql 均 Healthy													
前端与健康接口	首页 HTTP 200; /api/health 返回 ok													
数据库	真实 MySQL; 3 个版本迁移已应用													
Kind	3 个工作负载均 1/1; 单体入口 :30080													
回归	API 32/32; 浏览器 E2E 6/6													
Git 标签 保留原	远程标签 monolith-start;辅助脚本 03_devops/scripts	已在 GitHub 仓库中保留原系统基线版本，并通过 Git Tag 对对应版本进行标记，便于后续微服务拆分过程中进行版本追溯与对比。												

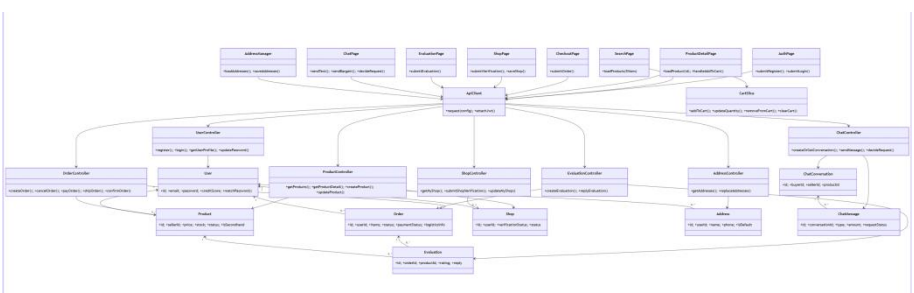
系统基线。	/tag-monolith.sh	GitHub Tags 页面: 
文档和图 三份说明书、三级模型和中期必备图。	02_docs/ 下需求、概要、详细设计说明书; 02_docs/images/ 中 40 张必需 SVG、ORDER-STATE、MS-SPLIT;02_docs/models/保留Mermaid源。	项目已建立完整的文档与模型目录，对需求分析、系统设计及后续实现提供对应文档支撑。同时补充系统架构图、UML 模型及相关设计图，确保系统需求、设计与实现之间具有对应关系。 02_docs 文件目录:  01-REQ-USECASE.svg 需求层：用例模型 购物与二手交易平台总用例图 (UC01-UC12)  12-ARCH-COMP.svg

概要设计：系统架构/组件模型



22-DESIGN-CLASS.svg

详细设计：设计类模型



用例 (UC01-UC12) :

[12 用例清单与追溯表](#); [需求说明书](#); [概要设计说明书](#); [详细设计说明书](#); [42 张图清单](#)

三类测试

单元/组件/规则、接口/API、

浏览器E2E，另有微服务与性能测试。

backend/tests/、
frontend/tests/、
frontend/e2e/、

services/*/*.test.js、
04_tests/performance/、

04_tests/reports/、
tests/测试报告-小学期.md;覆盖 UC01-UC12。

前后端单体 218 项通过：前端 106、后端 112。

测试层级	最新结果
后端单元 / 安全 / 控制器	80 通过, API 父入口跳过 1 项
前端 Vitest	100/100, 17 个文件
单体真实 MySQL API	32/32
Playwright E2E	6/6, UC01-UC12
微服务静态 / 公共层	18/18
微服务隔离集成	22/22 (1 + 3 + 1 + 17)
微服务网关 API/E2E	15/15, 49 项公开业务 API
交付与实验证据门禁	18/18

按测试报告输出统计；微服务各层存在复用，不计入上述单体合计。

		前端覆盖率：语句 94.42%、分支 81.83%、函数 92.34%、行 94.42%；四项均超过 80% 门禁。verify:full 通过，Vite 构建 1891 个模块。 完整测试报告 最新成功流水线 #77																
容器化 前端、后端、数据库可由 Docker /Compose 启动。	backend/Dockerfile frontend/Dockerfile 03_devops/ docker-compose.yml 启动命令： docker compose --env-file .env -f 03_devops/docker-compose.yml up -d --build --wait 完整核查： npm run verify:full	容器化验收结果（2026-09-02） 单体与微服务两种部署均已实测；前端、后端和数据库独立容器化。最新完整 CI/CD 已完成七个版本化镜像构建与 Kind 部署。 <table><tr><th>部署形态</th><th>组成 / 状态</th></tr><tr><td>单体 Compose</td><td>frontend、backend、mysql: 3 个容器均 Healthy</td></tr><tr><td>微服务 Compose</td><td>MySQL、3 个业务服务、API Gateway、前端：全部 Healthy</td></tr><tr><td>Kubernetes</td><td>单体 3 个、微服务 6 个工作负载均 1/1；健康、就绪、版本检查通过</td></tr></table> 换机启动 先执行 npm run env:init 生成本地配置，再运行： docker compose --env-file .env -f 03_devops/docker-compose.yml up -d --build --wait 数据库与健康检查 MySQL 首次启动执行初始化 SQL；后端在启动前完成版本化迁移。001-baseline-schema、002-marketplace-fields、003-bargain-redemption 均已应用。 首页 HTTP 200；/api/health 返回 status=ok、database=ok。单体提供 /api/live、/api/ready、/api/health、/api/version；Kind 使用持久卷并区分启动、存活和就绪探针。 证据 单体 Compose 数据库初始化 最新部署实测报告 CI/CD #77	部署形态	组成 / 状态	单体 Compose	frontend、backend、mysql: 3 个容器均 Healthy	微服务 Compose	MySQL、3 个业务服务、API Gateway、前端：全部 Healthy	Kubernetes	单体 3 个、微服务 6 个工作负载均 1/1；健康、就绪、版本检查通过								
部署形态	组成 / 状态																	
单体 Compose	frontend、backend、mysql: 3 个容器均 Healthy																	
微服务 Compose	MySQL、3 个业务服务、API Gateway、前端：全部 Healthy																	
Kubernetes	单体 3 个、微服务 6 个工作负载均 1/1；健康、就绪、版本检查通过																	
原系统 CI/CD 测试失败时阻断镜像构建和部署。	.github/workflows/ci-cd.yml 和 Actions 成功运行记录；验证记录说明后端测试、前端构建、E2E、微服务测试通过后才进入 Docker/Kubernetes。	GitHub Actions: softw-ci-cd #77 Success · main · c1b73ec · 2026-09-02 总时长 6 分 12 秒，12 个工件。 <table><tr><th>阶段</th><th>验证结果</th></tr><tr><td>后端 / API</td><td>单元 80 通过；真实 API 32/32</td></tr><tr><td>前端 / 覆盖率 / 构建</td><td>Vitest 100/100；覆盖率门禁与构建通过</td></tr><tr><td>浏览器 E2E</td><td>单体入口 6/6；微服务入口 6/6</td></tr><tr><td>微服务测试</td><td>4 个服务矩阵作业成功；网关 API/E2E 通过</td></tr><tr><td>安全检查</td><td>高危 / 严重依赖漏洞为 0</td></tr><tr><td>镜像发布</td><td>7 个镜像，使用完整 github.sha 标签</td></tr><tr><td>Kubernetes</td><td>Kind 部署、rollout、健康 / 就绪 / 版本通过</td></tr></table> 打开 #77 完整结果与工件流水线配置	阶段	验证结果	后端 / API	单元 80 通过；真实 API 32/32	前端 / 覆盖率 / 构建	Vitest 100/100；覆盖率门禁与构建通过	浏览器 E2E	单体入口 6/6；微服务入口 6/6	微服务测试	4 个服务矩阵作业成功；网关 API/E2E 通过	安全检查	高危 / 严重依赖漏洞为 0	镜像发布	7 个镜像，使用完整 github.sha 标签	Kubernetes	Kind 部署、rollout、健康 / 就绪 / 版本通过
阶段	验证结果																	
后端 / API	单元 80 通过；真实 API 32/32																	
前端 / 覆盖率 / 构建	Vitest 100/100；覆盖率门禁与构建通过																	
浏览器 E2E	单体入口 6/6；微服务入口 6/6																	
微服务测试	4 个服务矩阵作业成功；网关 API/E2E 通过																	
安全检查	高危 / 严重依赖漏洞为 0																	
镜像发布	7 个镜像，使用完整 github.sha 标签																	
Kubernetes	Kind 部署、rollout、健康 / 就绪 / 版本通过																	

CI/CD 完整门禁与证据

push 后执行后端/API、前端单元与覆盖率、浏览器、微服务与安全检查；全部通过后构建 7 个 SHA 镜像，再部署 Kind 并执行健康 / 就绪 / 版本检查。

docker-build 依赖全部测试与安全作业，kubernetes-deploy 依赖 docker-build；任何前置测试失败都阻断镜像和部署。

[最新成功 #77](#) [失败阻断证据 #53](#) [工作流](#)

微服务拆分方案

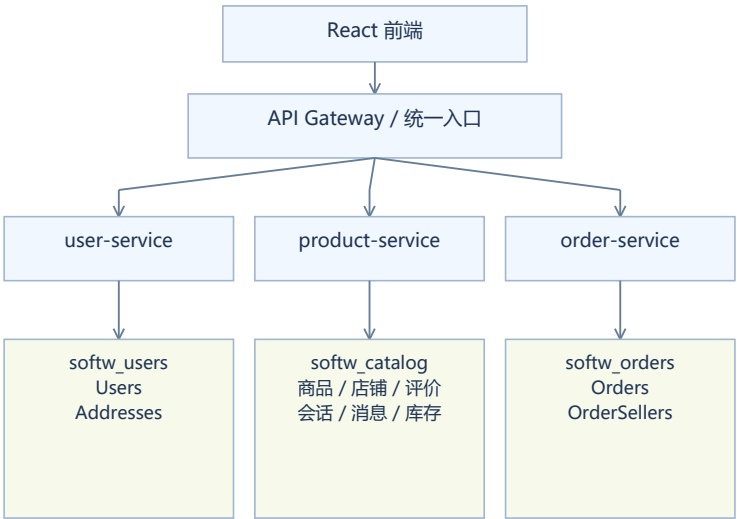
说明服务边界、接口和数据归属。

02_docs/微服务拆分设计.md、services/service-

boundaries.md、02_docs/微服务接口与数据归属.md；建议演示顺序：MS-SPLIT 图、接口清单、数据表归属、Actions 成功证明。

在保留原系统核心业务功能的基础上，项目已完成微服务拆分方案设计。按照业务领域对系统功能进行划分，并通过 API Gateway 统一对外提供访问入口，各业务服务之间按照既定接口进行协作，为后续独立部署、扩展及维护提供基础。

MS-SPLIT 微服务架构图：



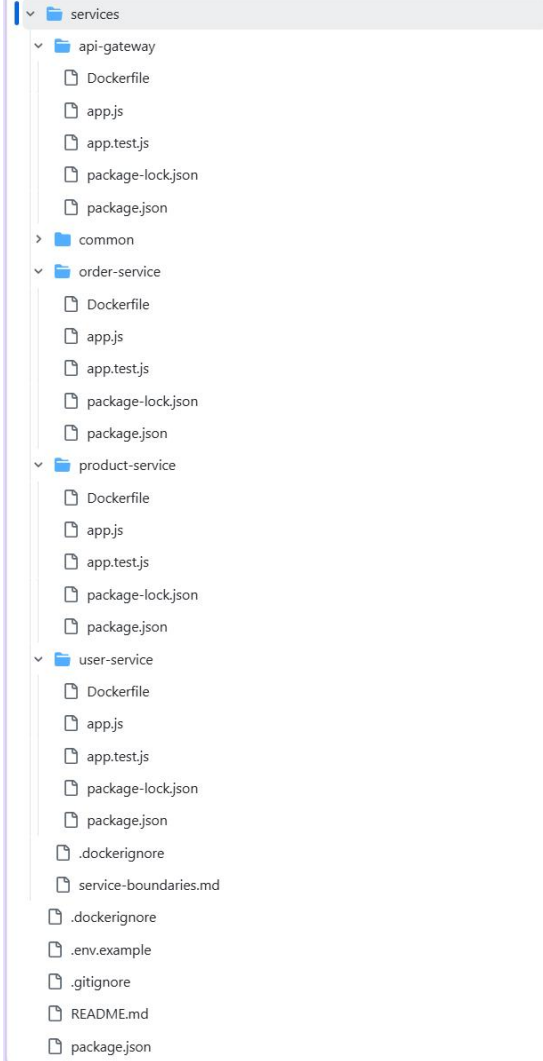
3 个业务服务 / 3 个独立 Schema
服务间经内部 HTTP API 交互，不跨库联表

微服务部署与回归实测

验证项	结果
独立服务	user、product、order 各自构建、测试、部署
静态 / 公共层	18/18
隔离集成	22/22 (user 1、product 3、order 1、gateway 17)
公开 API / 业务回归	15/15；49 项 API，UC01-UC12
页面 E2E	6/6
Kind	MySQL、3 个业务服务、网关、前端均 1/1

[最新流水线 #77](#) [逐项 API 映射](#)

二、微服务专项核对

专 项	核 查 要 点	证 据 位 置										
三 个 业 务 微 服 务	user-service : 用户/地址/角色/信用; product-service : 商品、二手、店铺、评价、聊天、上传、库存; order-service : 下单、支付、取消、发货、收货和状态机。API Gateway不计入业务服务数量。	系统已按照业务领域完成服务拆分, 形成 user-service、product-service 和 order-service 三个独立业务微服务, 分别负责用户、商品及订单相关业务; API Gateway 作为统一访问入口, 不计入业务服务数量。 										
网 关 与 端 点	网关按路径转发。四个后端组件统一提供: /live: 存活 /ready: 依赖就绪 /health: 健康摘要 /version: 版本 透传 JWT、Content-Type、Idempotency-Key、X-Request-Id; 依赖超时返回 503。	API Gateway 作为微服务统一访问入口, 根据请求路径将请求转发至对应业务服务, 并提供统一的健康检查、就绪检查及版本接口。相关 API 接口已通过自动化测试进行验证。 <table><tr><th>公开路径</th><th>目标</th></tr><tr><td>/api/users、/api/addresses</td><td>user-service</td></tr><tr><td>/api/products、/api/secondhand、/api/shops</td><td>product-service</td></tr><tr><td>/api/evaluations、/api/chats、/api/uploads、/uploads</td><td>product-service</td></tr><tr><td>/api/orders</td><td>order-service</td></tr></table> /live、/ready、/health、/version 已区分; 跨服务传递 X-Request-Id。依赖失败返回 503, 不伪造业务成功。	公开路径	目标	/api/users、/api/addresses	user-service	/api/products、/api/secondhand、/api/shops	product-service	/api/evaluations、/api/chats、/api/uploads、/uploads	product-service	/api/orders	order-service
公开路径	目标											
/api/users、/api/addresses	user-service											
/api/products、/api/secondhand、/api/shops	product-service											
/api/evaluations、/api/chats、/api/uploads、/uploads	product-service											
/api/orders	order-service											

数据归属	三个独立数据库： softw_users 、 softw_catalog 、 softw_orders ； 订单保存商品、地址和物流快照；服务之间通过内部 HTTP API交互 ， 不跨库联表。	微服务采用独立数据库进行数据隔离，用户、商品和订单数据分别归属于 softw_users 、 softw_catalog 和 softw_orders。服务之间通过内部 HTTP API 进行业务协作，不采用跨库联表方式。 数据库 / 表归属 <table><tr><th>数据库 / 文件卷</th><th>唯一管理服务与数据</th></tr><tr><td>softw_users</td><td>user-service: Users、Addresses</td></tr><tr><td>softw_catalog</td><td>product-service: Products、Shops、Evaluations、ChatConversations、ChatMessages、InventoryReservations</td></tr><tr><td>softw_orders</td><td>order-service: Orders、OrderSellers</td></tr><tr><td>uploads/</td><td>product-service 管理上传文件；其他服务保存 URL</td></tr></table> OrderSellers 保存订单与卖家的本地查询映射，用于索引、计数和分页；不读取其他服务的数据库。 订单保存商品、地址和物流快照。所有跨服务数据访问通过内部 HTTP API，不跨库联表。 订单数据模型与查询实现	数据库 / 文件卷	唯一管理服务与数据	softw_users	user-service: Users、Addresses	softw_catalog	product-service: Products、Shops、Evaluations、ChatConversations、ChatMessages、InventoryReservations	softw_orders	order-service: Orders、OrderSellers	uploads/	product-service 管理上传文件；其他服务保存 URL														
数据库 / 文件卷	唯一管理服务与数据																									
softw_users	user-service: Users、Addresses																									
softw_catalog	product-service: Products、Shops、Evaluations、ChatConversations、ChatMessages、InventoryReservations																									
softw_orders	order-service: Orders、OrderSellers																									
uploads/	product-service 管理上传文件；其他服务保存 URL																									
一致性与失败处理	订单通过商品服务库存预留、释放、完成接口维护库存；库存不足不创建订单；释放接口按 reservation Id 幂等；内部接口要求 X-Internal-Token。	订单流程通过商品服务提供的库存预留、释放及完成接口维护库存状态，并针对库存不足、订单取消及重复请求等异常场景进行验证，确保跨服务业务流程具有一致性和必要的幂等处理能力。 🔗 5. 跨服务调用与失败处理 <table><tr><th>调用</th><th>成功行为</th><th>失败处理</th></tr><tr><td>order-service -> product-service：库存预留</td><td>原子锁定商品行、扣减库存、写入幂等预留记录</td><td>商品不存在、不可售或库存不足时不创建订单；超时返回 503</td></tr><tr><td>order-service -> product-service：库存释放</td><td>取消订单或订单落库失败后恢复库存</td><td>释放接口按 reservationId 幂等，重复调用不重复增加库存</td></tr><tr><td>order-service -> product-service：交易完成</td><td>将预留标记完成，二手零库存商品转已售出</td><td>调用失败时订单不进入已完成，允许重试</td></tr><tr><td>product-service -> order-service：评价/退款资格</td><td>校验买家、商品和订单状态</td><td>校验失败不创建评价或退款申请</td></tr><tr><td>product/order-service -> user-service：身份</td><td>读取最小用户快照</td><td>用户服务不可用时拒绝需要身份的写操作；公开商品查询仍可用</td></tr><tr><td>product/order-service -> user-service：信用</td><td>按评价或履约结果更新信用</td><td>非核心信用更新失败不回滚已完成物流，记录为可重试调用</td></tr><tr><td>Gateway -> 业务服务</td><td>3 秒内返回真实上游结果</td><td>超时或连接失败返回 503，并注明失败路由</td></tr></table> 内部接口必须携带 X-Internal-Token。外部客户端只能通过网关和公开业务接口访问，不能调用库存、信用和购买证明接口。	调用	成功行为	失败处理	order-service -> product-service：库存预留	原子锁定商品行、扣减库存、写入幂等预留记录	商品不存在、不可售或库存不足时不创建订单；超时返回 503	order-service -> product-service：库存释放	取消订单或订单落库失败后恢复库存	释放接口按 reservationId 幂等，重复调用不重复增加库存	order-service -> product-service：交易完成	将预留标记完成，二手零库存商品转已售出	调用失败时订单不进入已完成，允许重试	product-service -> order-service：评价/退款资格	校验买家、商品和订单状态	校验失败不创建评价或退款申请	product/order-service -> user-service：身份	读取最小用户快照	用户服务不可用时拒绝需要身份的写操作；公开商品查询仍可用	product/order-service -> user-service：信用	按评价或履约结果更新信用	非核心信用更新失败不回滚已完成物流，记录为可重试调用	Gateway -> 业务服务	3 秒内返回真实上游结果	超时或连接失败返回 503，并注明失败路由
调用	成功行为	失败处理																								
order-service -> product-service：库存预留	原子锁定商品行、扣减库存、写入幂等预留记录	商品不存在、不可售或库存不足时不创建订单；超时返回 503																								
order-service -> product-service：库存释放	取消订单或订单落库失败后恢复库存	释放接口按 reservationId 幂等，重复调用不重复增加库存																								
order-service -> product-service：交易完成	将预留标记完成，二手零库存商品转已售出	调用失败时订单不进入已完成，允许重试																								
product-service -> order-service：评价/退款资格	校验买家、商品和订单状态	校验失败不创建评价或退款申请																								
product/order-service -> user-service：身份	读取最小用户快照	用户服务不可用时拒绝需要身份的写操作；公开商品查询仍可用																								
product/order-service -> user-service：信用	按评价或履约结果更新信用	非核心信用更新失败不回滚已完成物流，记录为可重试调用																								
Gateway -> 业务服务	3 秒内返回真实上游结果	超时或连接失败返回 503，并注明失败路由																								
部署与回归	Compose 可启动完整微服务拓扑； Kubernetes清单包含 MySQL PVC、三个业务服务、网关、前端和 HPA；网关公开 API/E2E 覆盖 UC01-UC12。	微服务版本已完成容器化部署，并通过自动化流水线验证镜像构建、Kubernetes 部署及服务健康状态。部署后进一步执行 API 与浏览器端回归测试，验证主要业务流程在微服务架构下能够正常运行。 部署后验收（2026-09-02） <table><tr><th>项目</th><th>已记录结果</th></tr><tr><td>微服务命名空间</td><td>softw-microservices: 6 个工作负载均 1/1</td></tr><tr><td>网关 API / 页面</td><td>15/15; 6/6; 覆盖 UC01-UC12</td></tr><tr><td>商品服务 HPA</td><td>min=1、max=5、CPU=60%; 实测 1 → 3 → 5 → 1</td></tr><tr><td>故障隔离 / 恢复</td><td>商品 503、订单依赖检查 206; 恢复后 HTTP 200</td></tr></table> Kind 部署原始记录 最新完整复核 CI/CD #77	项目	已记录结果	微服务命名空间	softw-microservices: 6 个工作负载均 1/1	网关 API / 页面	15/15; 6/6; 覆盖 UC01-UC12	商品服务 HPA	min=1、max=5、CPU=60%; 实测 1 → 3 → 5 → 1	故障隔离 / 恢复	商品 503、订单依赖检查 206; 恢复后 HTTP 200														
项目	已记录结果																									
微服务命名空间	softw-microservices: 6 个工作负载均 1/1																									
网关 API / 页面	15/15; 6/6; 覆盖 UC01-UC12																									
商品服务 HPA	min=1、max=5、CPU=60%; 实测 1 → 3 → 5 → 1																									
故障隔离 / 恢复	商品 503、订单依赖检查 206; 恢复后 HTTP 200																									

三、现场命令与记录

三、现场命令与记录（续）

本地快速核查

先启动 Docker Desktop；以下命令在仓库根目录执行，环境变量由本地 .env 提供，不将密钥写入材料。

目标	命令
环境初始化	npm run env:init
完整基础门禁	npm run verify:full
单体启动	docker compose --env-file .env -f 03_devops/docker-compose.yml up -d --build --wait
微服务启动	docker compose --env-file .env -f 03_devops/docker-compose.microservices.yml up -d --build --wait
单体 / 微服务 API	npm run test:api npm run test:services:api
单体页面 E2E	npm run test:e2e
单体健康检查	curl --fail http://127.0.0.1:3001/api/health
微服务就绪检查	curl --fail http://127.0.0.1:8081/ready

远程 / 集群证据

[softw-ci-cd #77](#): main / c1b73ec / Success。全部测试、7 个 SHA 镜像、Kind 部署及健康 / 就绪 / 版本检查成功。浏览器单体和微服务入口各 6/6。

[2026-09-02 完整复核](#): 后端 80、前端 100、单体 API 32、E2E 6；微服务静态 18、隔离集成 22、网关 API 15。覆盖率 94.42% / 81.83% / 92.34% / 94.42%。

四、依据文件

- [小学期交付总览](#): 中期检查项目与入口。
- [用例清单与追溯表](#): UC01-UC12 的需求、设计、代码和测试映射。
- [图表清单](#): 40 张必需图 + 2 张补充图，共 42 张 SVG / 42 个 Mermaid 源文件。
- [测试索引](#)、[最新测试与部署报告](#): 分层结果、覆盖率、环境与命令。
- [微服务公开 API 测试映射](#): 49 项公开业务接口与 UC01-UC12。
- [微服务接口与数据归属](#)、[订单服务实现](#): 服务边界、Orders / OrderSellers、内部交互与失败处理。
- [Kind 原始验收记录](#)、[HPA 与故障实验](#): 部署、扩缩容、降级与恢复。